

Eventra

Installation & Deployment Guide

Local Development on MacBook

Production Deployment on Azure CyberPanel

Version 1.0 | May 2026

Next.js 16 | Prisma | SQLite | Tailwind CSS

Table of Contents

1. Project Overview

Architecture, tech stack, and folder structure

2. Prerequisites

Software requirements for local and server environments

3. Local Setup on MacBook

Step-by-step installation and development workflow

4. Seeding the Database

Populating initial content for the website

5. Admin Panel Access

Logging in and managing website content

6. Deployment on Azure CyberPanel

Production server setup with PM2 and Nginx

7. Environment Variables

Configuration reference for all environments

8. Troubleshooting

Common issues and their solutions

9. Security Checklist

Production security hardening guide

1. Project Overview

Eventra is a full-stack event management website built with modern web technologies. It features a public-facing multi-page website with a Rose/Gold/Champagne color palette targeting the Nepali market, along with a comprehensive admin panel for managing all website content dynamically without touching code. The platform supports wedding planning, corporate events, and private party services, complete with portfolio galleries, testimonials, pricing packages, and an inquiry management system.

1.1 Tech Stack

Technology	Purpose	Version
Next.js	Full-stack React framework (App Router)	16.x
TypeScript	Type-safe JavaScript	5.x
Tailwind CSS	Utility-first CSS framework	4.x
shadcn/ui	UI component library	Latest
Prisma ORM	Database ORM	6.x
SQLite	Embedded database	3.x
Node.js	JavaScript runtime	18+ / 20+
Bun (optional)	Fast JS runtime alternative	1.x

1.2 Project Folder Structure

Understanding the project structure is essential for making modifications and troubleshooting issues. Below is a summary of the key directories and their purposes within the Eventra project.

```
eventra/  
  prisma/  
    schema.prisma      # Database models  
  public/  
    images/             # Static images  
    uploads/           # Admin uploads  
  src/  
    app/  
      page.tsx          # Hash-based router  
      layout.tsx        # Root layout  
      globals.css       # Brand colors  
      api/              # REST API routes  
        hero/route.ts  
        services/route.ts  
        portfolio/route.ts
```

```

testimonials/route.ts
pricing/route.ts
about/route.ts
contact-info/route.ts
inquiries/route.ts
admin/login/route.ts
admin/verify/route.ts
seed/route.ts
upload/route.ts
settings/route.ts
components/
  navbar.tsx          # Navigation bar
  footer.tsx          # Footer component
  pages/              # Public pages
  admin/              # Admin components
  ui/                 # shadcn/ui components
lib/
  db.ts               # Prisma client
  api.ts              # Frontend API helpers
  types.ts            # TypeScript types
  utils.ts            # Utility functions
db/
  custom.db           # SQLite database file
.env                  # Environment variables
package.json          # Dependencies & scripts

```

2. Prerequisites

Before installing Eventra, ensure your system meets the following software requirements. The project relies on Node.js for running the Next.js development server and build process, a package manager (npm or bun) for dependency installation, and Git for version control. Prisma CLI is used for database schema management and comes bundled as a project dependency.

2.1 For Local MacBook Development

Software	Minimum Version	Recommended Version	Install Method
Node.js	18.x	20.x LTS	brew install node or nvm install 20
npm	9.x	10.x	Comes with Node.js
Git	2.x	Latest	brew install git
Bun (optional)	1.x	Latest	curl -fsSL https://bun.sh/install bash

2.2 For Azure CyberPanel Server

Software	Minimum Version	Purpose	Install Method
Node.js	18.x	Runtime for Next.js	nvm install 20 or NodeSource repo
npm	9.x	Package manager	Comes with Node.js
PM2	5.x	Process manager	npm install -g pm2
Nginx	1.18+	Reverse proxy	CyberPanel includes this
Git	2.x	Code deployment	apt install git

Note: CyberPanel comes with Nginx (OpenLiteSpeed) pre-installed. You may use either Nginx or OpenLiteSpeed as the reverse proxy. This guide covers the Nginx approach as it is more commonly documented and widely supported by the community.

3. Local Setup on MacBook

This section walks you through the complete process of setting up the Eventra project on your MacBook for local development. The process involves cloning the project, installing dependencies, configuring the database, and starting the development server. Each step is designed to be executed in your Terminal application.

3.1 Step 1 - Open Terminal and Navigate to Your Projects Directory

Open the Terminal application on your MacBook (you can find it in Applications > Utilities > Terminal, or search using Spotlight with Cmd+Space). Navigate to the directory where you want to store the project. A common choice is the Projects folder in your home directory.

```
cd ~/Projects
```

3.2 Step 2 - Extract or Clone the Project

If you downloaded the project as a ZIP archive, extract it to your desired location. If using Git, clone the repository instead. The project folder should be named "my-project" or "eventra" depending on how it was packaged.

```
# If downloaded as ZIP:
unzip eventra.zip -d ~/Projects/eventra
cd ~/Projects/eventra

# If using Git:
git clone <your-repo-url> ~/Projects/eventra
cd ~/Projects/eventra
```

3.3 Step 3 - Install Node.js (if not already installed)

Node.js is the runtime that powers Next.js. The recommended approach for managing Node.js versions on macOS is using nvm (Node Version Manager), which allows you to install and switch between multiple Node.js versions easily. Alternatively, you can install Node.js directly via Homebrew.

```
# Option A: Using nvm (recommended)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh |
bash
source ~/.zshrc
nvm install 20
nvm use 20

# Option B: Using Homebrew
brew install node@20
```

After installation, verify Node.js and npm are available by running the following commands. You should see version numbers printed in the terminal output confirming the installation was successful.

```
node --version    # Should show v20.x.x
npm --version     # Should show 10.x.x
```

3.4 Step 4 - Install Project Dependencies

Navigate to the project root directory and install all the npm dependencies. This step downloads all the packages listed in package.json, including Next.js, React, Prisma, Tailwind CSS, shadcn/ui components, and all other libraries the project requires. This process may take 1-3 minutes depending on your internet connection speed.

```
cd ~/Projects/eventra
npm install
```

Note: If you have Bun installed, you can use "bun install" instead of "npm install" for faster installation. Bun is a drop-in replacement that typically installs dependencies 3-5 times faster than npm.

3.5 Step 5 - Configure Environment Variables

The project uses a .env file at the root to store configuration. The most critical variable is DATABASE_URL, which tells Prisma where to find the SQLite database file. Since SQLite uses a file path as the connection string, you need to update this path to match your local project location.

```
# Create or edit the .env file in the project root:
nano .env

# Add the following line (adjust the path to your actual project location):
DATABASE_URL=file:/Users/YOUR_USERNAME/Projects/eventra/db/custom.db
```

Replace "YOUR_USERNAME" with your actual macOS username. The database file "custom.db" will be created automatically when you run the Prisma push command in the next step. You do not need to create it manually.

3.6 Step 6 - Initialize the Database

Prisma needs to create the database schema and generate the client library. The "db push" command reads the Prisma schema file and creates all the necessary tables in the SQLite database. The "generate" command creates the TypeScript client that your application code uses to interact with the database.

```
npx prisma db push
npx prisma generate
```

After running these commands, you should see a "custom.db" file in the "db/" directory, and the Prisma client will be generated in "node_modules/@prisma/client". The terminal output should confirm that the schema was successfully pushed and the client was generated without errors.

3.7 Step 7 - Seed the Database with Initial Content

The project includes a seed API that populates the database with default content including hero slides, services, portfolio items, testimonials, pricing packages, contact information, and site settings. You need to start the dev server first, then call the seed endpoint.

```
# Start the development server
npm run dev

# In a separate terminal window, seed the database:
curl -X POST http://localhost:3000/api/seed
```

You should see a response: {"message":"Database seeded successfully!"}. This only needs to be done once. If the database already has content, the seed endpoint safely skips re-seeding without duplicating data, so it is safe to call it multiple times.

3.8 Step 8 - Access the Website

With the development server running, open your web browser and navigate to the following URLs to access different parts of the application. The development server supports hot module replacement (HMR), so any changes you make to the source code will be reflected in the browser automatically without needing to restart the server.

Page	URL	Description
Homepage	http://localhost:3000	Hero carousel, services, testimonials, CTA
About	http://localhost:3000#/about	Company story and features
Services	http://localhost:3000#/services	Wedding, Corporate, Parties
Portfolio	http://localhost:3000#/portfolio	Filterable image gallery
Testimonials	http://localhost:3000#/testimonials	Client reviews
Pricing	http://localhost:3000#/pricing	3-tier pricing packages
Contact	http://localhost:3000#/contact	Inquiry form + WhatsApp/Viber

Admin Login	http://localhost:3000#/admin/login	Admin panel authentication
Admin Dashboard	http://localhost:3000#/admin	Content management home

Note: The website uses hash-based routing (e.g., [#/about](#) instead of [/about](#)). This means all pages are served from the single `page.tsx` entry point, and navigation happens client-side without full page reloads. Bookmarking any hash URL will work correctly when you revisit it.

4. Seeding the Database

Database seeding is the process of populating your database with initial data. The Eventra project includes a comprehensive seed endpoint that creates all the default content needed for the website to display properly. Without seeding, the website will show loading skeletons instead of actual content. This section explains what data gets seeded and how to manage the seeding process.

4.1 What Gets Seeded

Data Type	Count	Details
Hero Slides	3	Rotating banner images with titles and subtitles
About Content	1	Company description, features, and images
Services	3	Wedding Planning, Corporate Events, Private Parties
Portfolio Items	9	Gallery images across 3 categories
Testimonials	4	Client reviews with star ratings
Pricing Packages	3	Basic (NPR 25K), Standard (NPR 55K), Premium (NPR 95K)
Contact Info	1	Phone, email, address, WhatsApp, Viber
Site Settings	2	Brand name and tagline

4.2 Resetting the Database

If you need to start fresh with a clean database (for example, after making schema changes or if the data becomes corrupted), you can reset the database completely. This will delete all existing data and recreate the schema from scratch. After resetting, you must run the seed endpoint again to populate the database with default content.

```
# Delete the existing database file
rm db/custom.db

# Recreate the schema
npx prisma db push
```



```
npx prisma generate

# Restart the dev server, then seed:
npm run dev
curl -X POST http://localhost:3000/api/seed
```

5. Admin Panel Access

The admin panel allows you to manage all website content through a user-friendly interface without needing to modify any code or database entries directly. You can add, edit, and delete hero slides, services, portfolio items, testimonials, pricing packages, contact information, and site settings. The admin panel also provides a dashboard for viewing and managing customer inquiries submitted through the contact form.

5.1 Login Credentials

Field	Value
URL	http://localhost:3000#/admin/login
Email	admin@eventra.com
Password	admin123

Note: The default credentials are hardcoded for development purposes. For production deployment, you **MUST** change these credentials by modifying the file `src/app/api/admin/login/route.ts`. This is a critical security requirement before making the site publicly accessible.

5.2 Admin Panel Sections

Once logged in, the admin panel provides access to the following sections. Each section allows full CRUD (Create, Read, Update, Delete) operations on the corresponding data. Changes made in the admin panel are immediately reflected on the public website, so there is no need to restart the server or rebuild the application.

- **Dashboard** - Overview of all content counts and recent inquiries
- **Hero Slides** - Manage the rotating banner images, titles, and subtitles on the homepage
- **Services** - Add, edit, or remove service offerings with descriptions, icons, and feature lists
- **Portfolio** - Upload and manage gallery images with category filtering
- **Testimonials** - Manage client reviews, star ratings, and display order
- **Pricing** - Edit pricing tiers, features, and "most popular" badge placement
- **Inquiries** - View, respond to, and manage customer inquiries from the contact form
- **Contact Info** - Update phone, email, address, WhatsApp, and Viber numbers

- **Settings** - Change brand name, tagline, and other site-wide settings

5.3 Image Uploads

The admin panel supports image uploads for hero slides, services, and portfolio items. Uploaded images are stored in the "public/uploads/" directory and served statically by Next.js. The supported formats include JPG, PNG, GIF, and WebP. For optimal performance, images should be resized to appropriate dimensions before uploading: hero slides should be approximately 1920x1080 pixels, service thumbnails around 800x600 pixels, and portfolio images between 800x600 and 1200x800 pixels.

6. Deployment on Azure CyberPanel

Deploying Eventra to an Azure Virtual Machine running CyberPanel involves building the Next.js application for production, setting up a process manager to keep the application running, and configuring a reverse proxy (Nginx or OpenLiteSpeed) to route web traffic to your application. This section provides a complete step-by-step guide for production deployment, covering server preparation through to going live.

6.1 Step 1 - Provision an Azure VM

Create an Azure Virtual Machine with the following recommended specifications. The application is lightweight thanks to SQLite and the standalone Next.js output, so a basic VM is sufficient for moderate traffic. For high-traffic scenarios, consider upgrading to a larger instance with more CPU cores and RAM.

Specification	Minimum	Recommended
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
CPU	1 vCPU	2 vCPU
RAM	1 GB	2 GB
Storage	20 GB SSD	40 GB SSD
CyberPanel	Free version	Free version

6.2 Step 2 - Install CyberPanel

If CyberPanel is not already installed on your Azure VM, you can install it using the official installer script. CyberPanel provides a web-based control panel that simplifies server management, including website creation, SSL certificate management, and email configuration. The installation process typically takes 10-20 minutes depending on server performance.

```
# SSH into your Azure VM
ssh azureuser@YOUR_VM_IP
```

```
# Run the CyberPanel installer
sh <(curl https://cyberpanel.net/install.sh || wget -O - https://cyberpanel.net/install.sh)
```

Follow the on-screen prompts to complete the CyberPanel installation. The installer will automatically set up OpenLiteSpeed, MariaDB, and other necessary components. After installation, access the CyberPanel dashboard at https://YOUR_VM_IP:8090 with the default admin credentials (admin / 1234567 - change this immediately after first login).

6.3 Step 3 - Install Node.js and PM2 on the Server

SSH into your server and install Node.js using nvm (Node Version Manager), which provides the most flexible and reliable Node.js installation on Linux servers. Then install PM2, a production-grade process manager that will keep your Next.js application running continuously, automatically restarting it if it crashes and providing monitoring capabilities.

```
# Install nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh |
bash
source ~/.bashrc

# Install Node.js 20
nvm install 20
nvm use 20
nvm alias default 20

# Install PM2 globally
npm install -g pm2
```

Verify the installations are successful by checking the version numbers. Both node and pm2 commands should return valid version strings confirming they are properly installed and accessible in your system PATH.

```
node --version      # v20.x.x
pm2 --version       # 5.x.x
```

6.4 Step 4 - Upload Project Files

Transfer the project files from your MacBook to the Azure server. The recommended approach is using Git for deployment, which provides version control and easy updates. Alternatively, you can use SCP or rsync for direct file transfer. The project should be placed in a dedicated directory on the server, such as /home/eventra/ or /var/www/eventra/.

```
# Option A: Using Git (recommended)
ssh azureuser@YOUR_VM_IP
mkdir -p /home/eventra
cd /home/eventra
git clone <your-repo-url> .
```

```
# Option B: Using SCP from your MacBook
scp -r ~/Projects/eventra azureuser@YOUR_VM_IP:/home/eventra/

# Option C: Using rsync (best for updates)
rsync -avz --exclude node_modules --exclude .next \
  ~/Projects/eventra/ azureuser@YOUR_VM_IP:/home/eventra/
```

6.5 Step 5 - Install Dependencies and Build

On the server, navigate to the project directory and install the production dependencies. Then build the Next.js application for production using the build script defined in package.json. The build process compiles TypeScript, optimizes CSS and JavaScript, and creates a standalone output in the .next/standalone/ directory that includes everything needed to run the application.

```
cd /home/eventra
npm install

# Configure environment
nano .env
# Set: DATABASE_URL=file:/home/eventra/db/custom.db

# Initialize database
npx prisma db push
npx prisma generate

# Build for production
npm run build
```

Note: The build command ("next build") also copies static files and the public directory into the standalone output. The standalone output is optimized for production and includes only the necessary files, resulting in a smaller deployment footprint compared to running the full development setup.

6.6 Step 6 - Seed the Database on the Server

After building the application, you need to seed the database with initial content. Start the server temporarily to call the seed endpoint, then stop it. This only needs to be done once during initial setup.

```
# Start the server temporarily in the background
NODE_ENV=production node .next/standalone/server.js &

# Wait a moment for the server to start, then seed:
sleep 5
curl -X POST http://localhost:3000/api/seed

# Stop the temporary server
kill %1
```

6.7 Step 7 - Configure PM2 Process Manager

PM2 ensures your Next.js application runs continuously in production, automatically restarting if it crashes and providing log management. Create a PM2 ecosystem configuration file that defines how your application should be run, including environment variables and startup settings.

```
# Create PM2 ecosystem file
cat << 'EOF' > /home/eventra/ecosystem.config.js
module.exports = {
  apps: [{
    name: 'eventra',
    script: '.next/standalone/server.js',
    cwd: '/home/eventra',
    env: {
      NODE_ENV: 'production',
      PORT: 3000,
      DATABASE_URL: 'file:/home/eventra/db/custom.db',
    },
  },
  instances: 1,
  autorestart: true,
  watch: false,
  max_memory_restart: '500M',
  error_file: '/home/eventra/logs/error.log',
  out_file: '/home/eventra/logs/out.log',
  log_date_format: 'YYYY-MM-DD HH:mm:ss',
}]
};
EOF

# Create logs directory
mkdir -p /home/eventra/logs

# Start the application with PM2
cd /home/eventra
pm2 start ecosystem.config.js

# Verify it is running
pm2 status
pm2 logs eventra --lines 20
```

To ensure PM2 automatically restarts your application when the server reboots, save the PM2 process list and generate a startup script. This creates a system service that launches PM2 and all its managed processes on system boot.

```
# Save PM2 process list
pm2 save

# Generate startup script (run as root or with sudo)
pm2 startup
# PM2 will output a command - copy and run that command
```

6.8 Step 8 - Configure Nginx Reverse Proxy

CyberPanel uses OpenLiteSpeed by default, but you can configure Nginx as a reverse proxy to forward web traffic to your Next.js application running on port 3000. If you prefer using OpenLiteSpeed directly, refer to the CyberPanel documentation for creating a proxy context. This guide covers the Nginx approach as it provides more flexibility and is easier to configure for custom applications.

```
# Install Nginx (if not already installed)
sudo apt update
sudo apt install nginx -y

# Create Nginx configuration
cat << 'EOF' | sudo tee /etc/nginx/sites-available/eventra
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 86400;
    }

    # Static files served directly by Nginx for better performance
    location /_next/static/ {
        alias /home/eventra/.next/static/;
        expires 365d;
        access_log off;
        add_header Cache-Control "public, immutable";
    }

    # Public assets served directly by Nginx
    location /images/ {
        alias /home/eventra/public/images/;
        expires 30d;
        access_log off;
    }

    location /uploads/ {
        alias /home/eventra/public/uploads/;
        expires 30d;
        access_log off;
    }

    # Client upload size limit
```

```

        client_max_body_size 10M;
    }
EOF

# Enable the site
sudo ln -s /etc/nginx/sites-available/eventra /etc/nginx/sites-enabled/

# Test configuration
sudo nginx -t

# Reload Nginx
sudo systemctl reload nginx

```

Note: Replace "yourdomain.com" and "www.yourdomain.com" with your actual domain name. If you do not have a domain yet, you can use the server IP address temporarily. Make sure your domain DNS A record points to the Azure VM IP address.

6.9 Step 9 - SSL Certificate with Let us Encrypt

Secure your website with HTTPS using a free SSL certificate from Let us Encrypt. Certbot automates the certificate issuance and renewal process. After installation, Certbot will automatically redirect all HTTP traffic to HTTPS, ensuring all communication between visitors and your website is encrypted.

```

# Install Certbot
sudo apt install certbot python3-certbot-nginx -y

# Obtain and install SSL certificate
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Certbot will automatically configure Nginx for HTTPS
# and set up auto-renewal via a cron job

# Verify auto-renewal is configured
sudo certbot renew --dry-run

```

6.10 Step 10 - Open Azure Firewall Ports

In the Azure Portal, ensure the following inbound ports are open in your VM Network Security Group (NSG). This is critical because even with correct Nginx configuration, the Azure firewall will block traffic if the required ports are not explicitly allowed.

Port	Protocol	Purpose	Source
80	TCP	HTTP traffic (redirects to HTTPS)	Internet
443	TCP	HTTPS traffic	Internet
22	TCP	SSH access for administration	Your IP only

8090	TCP	CyberPanel dashboard (optional)	Your IP only
------	-----	---------------------------------	--------------

Note: For security, restrict SSH (port 22) and CyberPanel (port 8090) access to your own IP address only. Never leave these ports open to all Internet traffic (0.0.0.0/0) as this exposes your server to brute force attacks and unauthorized access attempts.

7. Environment Variables

Environment variables are used to configure the application for different environments (development, production) without modifying the source code. All environment variables are stored in the `.env` file at the project root. This file should never be committed to version control as it contains sensitive configuration data.

Variable	Description	Example (Local)	Example (Production)
DATABASE_URL	SQLite connection string	file:/Users/you/eventra/db/custom.db	file:/home/eventra/db/custom.db
PORT	Server port (standalone only)	3000	3000
NODE_ENV	Environment mode	development	production

Note: The DATABASE_URL path must be an absolute path to the SQLite database file. On macOS, use `/Users/username/...` format. On Linux, use `/home/username/...` format. Relative paths are not supported by Prisma for SQLite connections.

8. Troubleshooting

This section covers common issues you may encounter during installation, development, or deployment, along with their solutions. Most issues can be resolved by following the steps outlined below. If you encounter an issue not covered here, check the terminal/console output for error messages as they often contain specific guidance about the root cause.

8.1 npm install Fails

If npm install fails with errors, it is usually due to network issues, cached corrupted packages, or Node.js version incompatibility. Try the following solutions in order, starting with the least invasive approach.

```
# Clear npm cache
npm cache clean --force

# Delete node_modules and reinstall
rm -rf node_modules package-lock.json
npm install
```



```
# If using Bun instead:  
rm -rf node_modules bun.lockb  
bun install
```

8.2 Prisma Client Not Found

If you see errors like "Cannot find module @prisma/client" or "PrismaClient is not a constructor", the Prisma client has not been generated or the database schema has not been pushed. This typically happens after a fresh clone or after modifying the Prisma schema.

```
# Regenerate Prisma client  
npx prisma generate  
  
# Push schema to database  
npx prisma db push
```

8.3 Blank Page / Only Logo Visible

If the website shows only a logo or blank page, the database has likely not been seeded with content. The page components fetch data from the API, and without seeded data, they display loading skeletons or fallback content. Ensure the dev server is running and seed the database.

```
# Make sure dev server is running  
npm run dev  
  
# Seed the database  
curl -X POST http://localhost:3000/api/seed
```

8.4 Port 3000 Already in Use

If you see an error like "Port 3000 is already in use", another process is occupying that port. You can either kill the existing process or start the dev server on a different port.

```
# Find and kill the process on port 3000  
lsof -i :3000  
kill -9 <PID>  
  
# Or use a different port  
npm run dev -- -p 3001
```

8.5 Images Not Displaying

If images are not loading on the website, verify that the public/ directory contains the expected image files (hero1.png, hero2.png, hero3.png, about1.png, about2.png, about3.png, wedding.png, corporate.png, party.png, portfolio1-6.png, logo-eventra.png). These images should have been included with the project download. If any are missing, the corresponding sections will show broken image icons.

8.6 Build Fails on Server

If the production build fails on the Azure server, common causes include insufficient memory (the build process can use 500MB+ of RAM), missing dependencies, or incorrect Node.js version. Try increasing swap space, verifying Node.js version, and ensuring all dependencies are installed.

```
# Check available memory
free -h

# Add swap space if RAM is limited (1GB swap)
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Verify Node.js version
node --version # Should be 18+ or 20+

# Reinstall dependencies and rebuild
rm -rf node_modules
npm install
npm run build
```

8.7 PM2 Application Keeps Restarting

If PM2 shows the application in "errored" or "stopped" state, check the error logs for details. Common causes include incorrect DATABASE_URL path, missing database file, or the standalone server.js not being found at the expected path.

```
# Check PM2 logs
pm2 logs eventra --lines 50

# Verify database exists
ls -la /home/eventra/db/custom.db

# Verify standalone build exists
ls -la /home/eventra/.next/standalone/server.js

# Restart with fresh state
pm2 delete eventra
pm2 start ecosystem.config.js
```

9. Security Checklist

Before making your Eventra website publicly accessible on the Internet, it is essential to implement these security measures. Failing to address these items could expose your server and user data to potential threats, unauthorized access, or data loss. Security is not a one-time task but an ongoing process of monitoring, updating, and hardening your deployment.

9.1 Critical Security Items

- **Change admin credentials** - Modify the hardcoded email/password in `src/app/api/admin/login/route.ts`. Use a strong, unique password and consider implementing environment variable-based credentials instead of hardcoding them in the source code.
- **Enable HTTPS** - Install an SSL certificate using Let us Encrypt (Certbot) as described in Section 6.9. Never serve a production website over plain HTTP, as it exposes user data including passwords and form submissions to interception.
- **Restrict SSH access** - Configure the Azure NSG to allow SSH connections only from your IP address. Disable password-based SSH authentication and use SSH keys instead for significantly stronger security.
- **Set NODE_ENV=production** - This disables Next.js development features, error stack traces in responses, and other debug information that should not be visible to end users in a production environment.
- **Protect the .env file** - Ensure the `.env` file is included in `.gitignore` and is never committed to version control. This file contains the database path and potentially other sensitive configuration.
- **Secure the /api/seed endpoint** - In production, consider removing or protecting the seed endpoint to prevent unauthorized database manipulation. Anyone who can access this endpoint could modify your website content.
- **Set up database backups** - SQLite stores everything in a single file (`custom.db`). Set up automated daily backups using cron and `rsync` or a cloud backup service. A simple backup script can copy the database file to a backup directory with a date-stamped filename.

9.2 Backup Script Example

```
#!/bin/bash
# Daily backup script for Eventra database
# Save as /home/eventra/backup.sh

DATE=$(date +%Y-%m-%d)
BACKUP_DIR="/home/eventra/backups"
mkdir -p $BACKUP_DIR

# Copy and compress the database
cp /home/eventra/db/custom.db $BACKUP_DIR/custom_${DATE}.db
gzip $BACKUP_DIR/custom_${DATE}.db

# Keep only last 30 days of backups
find $BACKUP_DIR -name "custom_*.db.gz" -mtime +30 -delete

echo "Backup completed: custom_${DATE}.db.gz"
```

Add this script to a daily cron job to automate the backup process. Run `"crontab -e"` and add the following line to execute the backup every day at 2 AM server time:

```
0 2 * * * /home/eventra/backup.sh >> /home/eventra/logs/backup.log 2>&1
```

9.3 Update Deployment Workflow

When you need to update the production website with new code changes, follow this deployment workflow to minimize downtime and ensure a smooth update process. The key principle is to build the new version before stopping the old one, so the switchover happens in seconds rather than minutes.

```
# SSH into the server
ssh azureuser@YOUR_VM_IP

# Navigate to the project
cd /home/eventra

# Pull latest changes
git pull origin main

# Install any new dependencies
npm install

# Rebuild for production
npm run build

# Restart PM2 (zero-downtime reload)
pm2 reload eventra

# Verify the application is running
pm2 status
pm2 logs eventra --lines 10
```